

PRUEBA TÉCNICA .NET · AUDISOFT

Gestión de Notas

Documento de instalación y ejecución

API REST .NET 10 · Angular 22 · SQL Server · Postman

Resumen

Aplicación web para administrar **estudiantes**, **profesores** y las **notas** que los profesores asignan a los estudiantes. Incluye un API REST en .NET 10 con SQL Server, un frontend en Angular 22 con menú, tablas paginadas y formularios de creación, edición y eliminación, alertas para cada acción, validaciones en el backend y una colección de Postman con pruebas automáticas.

Este documento explica cómo instalar y ejecutar el proyecto en un equipo local, paso a paso y con capturas de cada etapa. **No se necesita Docker ni ninguna cuenta en la nube.**

Fredy Alejandro Espinel Prado

Repositorio: github.com/alejo0001/gestion-notas-audisoft

Septiembre de 2026

Contenido

1. Descripción de la solución	3
Modelo de datos	3
Demostración en línea	3
Contenido de la entrega	4
2. Requisitos previos	5
3. Base de datos	6
Opción A — Scripts SQL	6
Opción B — Migraciones de Entity Framework Core	7
Cadena de conexión	7
4. API REST	8
Endpoints	9
5. Pruebas unitarias	9
6. Frontend	10
7. Uso de la aplicación	11
Estudiantes y profesores	12
Notas	13
Eliminación	14
8. Pruebas del API con Postman	16
9. Solución de problemas	17

1. Descripción de la solución

La solución sigue una arquitectura en capas que separa las reglas del negocio del acceso a datos y de la presentación:

Componente	Tecnología	Responsabilidad
Base de datos	SQL Server (LocalDB, Express o Azure SQL)	Tablas Estudiante, Profesor y Nota con llaves foráneas, restricción del valor de la nota e índices únicos.
API REST	.NET 10 · ASP.NET Core · EF Core 10	Cuatro capas (Domain, Application, Infrastructure, Api). Validación con FluentValidation, paginación en el servidor y errores estándar ProblemDetails (RFC 9457).
Frontend	Angular 22 · Angular Material · SweetAlert2	Menú lateral, panel de inicio y una página por recurso con tabla paginada, búsqueda, orden, formularios y alertas.
Pruebas	xUnit · Postman	Pruebas unitarias de los servicios y colección de Postman que prueba cada endpoint.

Modelo de datos

Profesor	(Id, Nombre)	1	—<	N	Nota.IdProfesor
Estudiante	(Id, Nombre)	1	—<	N	Nota.IdEstudiante
Nota	(Id, Nombre, IdProfesor, IdEstudiante, Valor)				

- **FK_Nota_Profesor** y **FK_Nota_Estudiante** con *ON DELETE NO ACTION*: no se puede eliminar un estudiante o profesor que tenga notas.
- **CK_Nota_Valor**: la nota debe estar entre 0,0 y 5,0 (DECIMAL(3,2), escala colombiana).
- Índices únicos: el nombre de estudiantes y profesores no se repite, y un estudiante no puede tener dos veces la misma evaluación con el mismo profesor.

Demostración en línea

Además de la ejecución local, la aplicación está publicada en Azure y se despliega automáticamente con GitHub Actions en cada cambio:

Recurso	URL
Aplicación	https://nice-river-0fe292b0f.4.azurestaticapps.net
API (Swagger)	https://ca-gestion-notas-api.yellowriver-838dad11.eastus2.azurecontainerapps.io/swagger
Repositorio	https://github.com/alejo0001/gestion-notas-audisoft

La primera carga de la demostración en línea puede tardar hasta un minuto: el API y la base de datos se apagan solos cuando no hay tráfico para no generar costos.

Contenido de la entrega

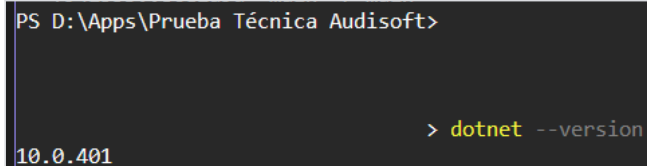
Carpeta	Contenido
backend/	Solución .NET GestionNotas.slnx: proyectos Domain, Application, Infrastructure, Api y UnitTests.
frontend/	Aplicación Angular.
database/	01_crear_base_datos.sql (estructura y restricciones) y 02_datos_prueba.sql (datos de ejemplo).
docs/postman/	Colección de Postman y entornos Local y Azure.
docs/decisions/, docs/specs/	Decisiones de arquitectura (ADR) y especificaciones funcionales.
README.md	Resumen del proyecto e instrucciones rápidas.

2. Requisitos previos

Software	Versión	Uso
.NET SDK	10.0 o superior	Compilar y ejecutar el API y las pruebas.
Node.js	22 LTS o superior (incluye npm)	Instalar dependencias y ejecutar el frontend.
SQL Server	LocalDB (viene con Visual Studio), Express o Developer	Base de datos. La configuración por defecto usa LocalDB.
Cliente SQL	Visual Studio, SQL Server Management Studio o Azure Data Studio	Ejecutar los scripts SQL (opción A del paso 3).
Postman	Versión actual	Opcional: ejecutar la colección de pruebas del API.

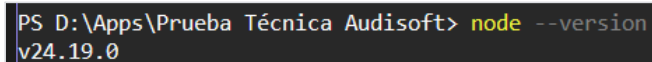
Para comprobar las versiones instaladas, en una terminal (PowerShell o CMD):

```
dotnet --version
node --version
```



```
PS D:\Apps\Prueba Técnica Audisoft>
> dotnet --version
10.0.401
```

Figura 1. Versión del SDK de .NET instalada (10.0.401).



```
PS D:\Apps\Prueba Técnica Audisoft> node --version
v24.19.0
```

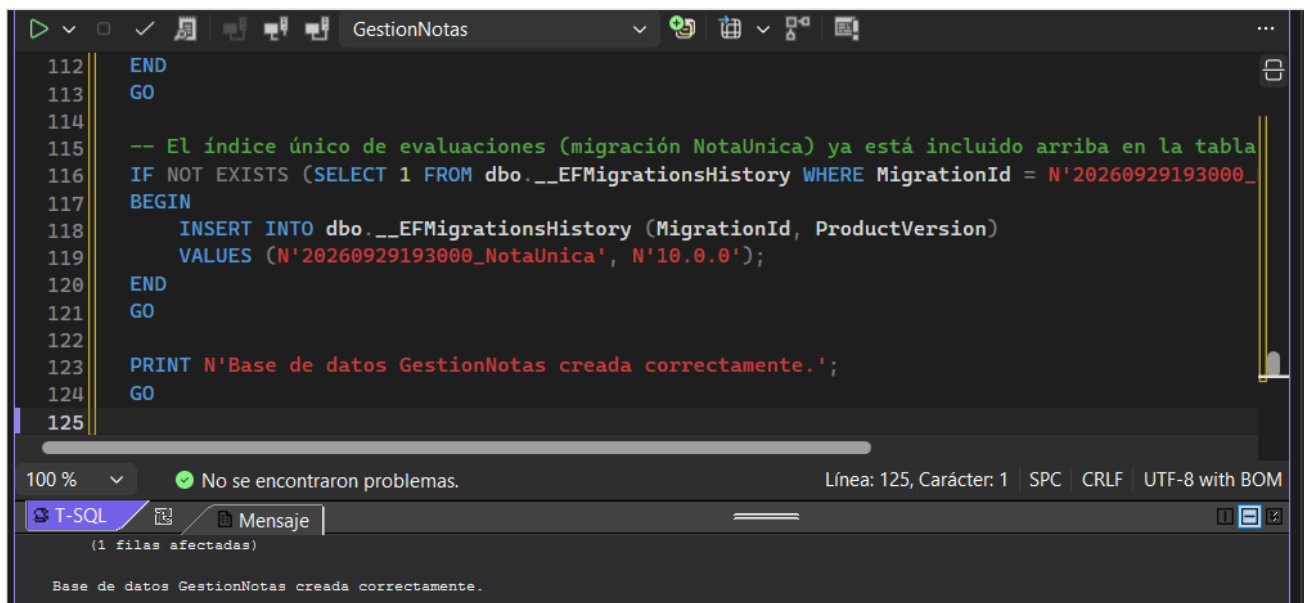
Figura 2. Versión de Node.js instalada (24.19.0).

3. Base de datos

Descomprima el ZIP en una carpeta (por ejemplo C:\GestionNotas). Para crear la base de datos elija **una** de las dos opciones; ambas dejan exactamente la misma estructura y los mismos datos de prueba.

Opción A — Scripts SQL

1. Abra su cliente SQL y conéctese a la instancia. Para LocalDB, el servidor es (localdb)\MSSQLLocalDB.
2. Con la base **master** seleccionada, abra y ejecute database/01_crear_base_datos.sql. Crea la base **GestionNotas**, las tablas, las llaves foráneas, la restricción del valor y los índices.
3. Abra y ejecute database/02_datos_prueba.sql. Inserta 5 profesores, 12 estudiantes y 28 notas.



```
112 END
113 GO
114
115 -- El índice único de evaluaciones (migración NotaUnica) ya está incluido arriba en la tabla
116 IF NOT EXISTS (SELECT 1 FROM dbo.__EFMigrationsHistory WHERE MigrationId = N'20260929193000_
117 BEGIN
118     INSERT INTO dbo.__EFMigrationsHistory (MigrationId, ProductVersion)
119     VALUES (N'20260929193000_NotaUnica', N'10.0.0');
120 END
121 GO
122
123 PRINT N'Base de datos GestionNotas creada correctamente.';
124 GO
125
```

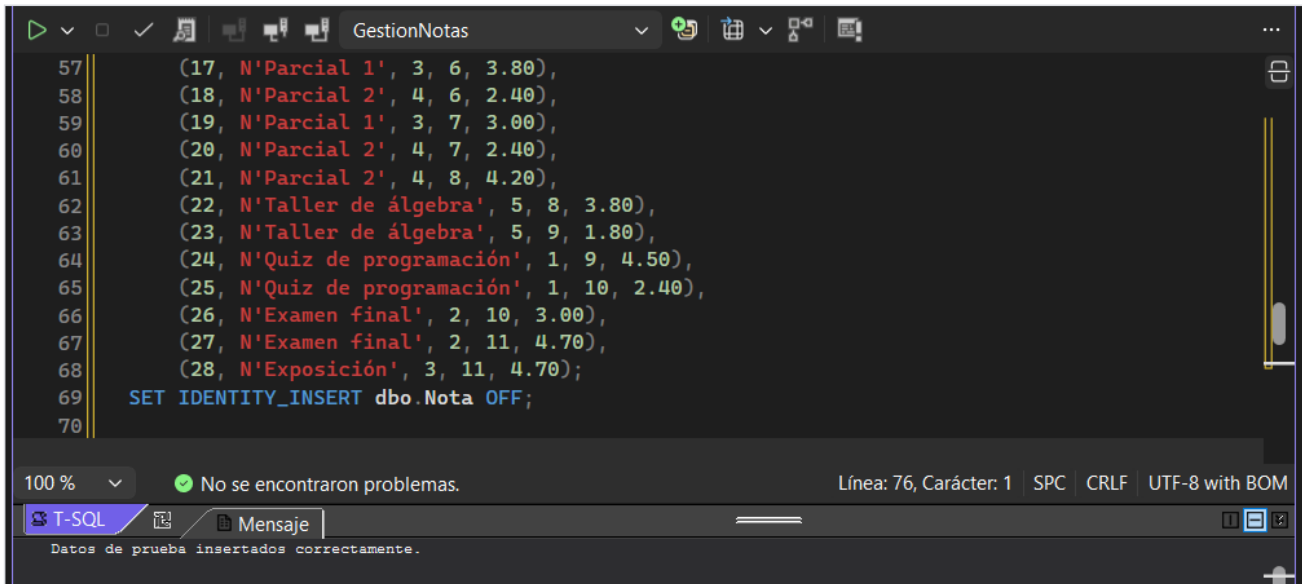
100 % No se encontraron problemas. Línea: 125, Carácter: 1 SPC CRLF UTF-8 with BOM

T-SQL Mensaje

(1 filas afectadas)

Base de datos GestionNotas creada correctamente.

Figura 3. Ejecución de 01_crear_base_datos.sql en Visual Studio: «Base de datos GestionNotas creada correctamente».



```

57 (17, N'Parcial 1', 3, 6, 3.80),
58 (18, N'Parcial 2', 4, 6, 2.40),
59 (19, N'Parcial 1', 3, 7, 3.00),
60 (20, N'Parcial 2', 4, 7, 2.40),
61 (21, N'Parcial 2', 4, 8, 4.20),
62 (22, N'Taller de álgebra', 5, 8, 3.80),
63 (23, N'Taller de álgebra', 5, 9, 1.80),
64 (24, N'Quiz de programación', 1, 9, 4.50),
65 (25, N'Quiz de programación', 1, 10, 2.40),
66 (26, N'Examen final', 2, 10, 3.00),
67 (27, N'Examen final', 2, 11, 4.70),
68 (28, N'Exposición', 3, 11, 4.70);
69 SET IDENTITY_INSERT dbo.Nota OFF;
70

```

100 % No se encontraron problemas. Línea: 76, Carácter: 1 SPC CRLF UTF-8 with BOM

T-SQL Mensaje

Datos de prueba insertados correctamente.

Figura 4. Ejecución de 02_datos_prueba.sql: «Datos de prueba insertados correctamente».

Los scripts registran las migraciones equivalentes de Entity Framework en la tabla `__EFMigrationsHistory`. Así, al arrancar el API después, EF Core reconoce que el esquema ya existe y no intenta crearlo de nuevo.

Opción B — Migraciones de Entity Framework Core

Desde la carpeta backend, crea la base con la estructura y los datos de prueba:

```

cd backend
dotnet tool restore
dotnet ef database update -p src/GestionNotas.Infraestructura -s src/GestionNotas.Api

```

Además, al ejecutar el API en modo *Development* se aplican automáticamente las migraciones pendientes, así que este paso también puede omitirse.

Cadena de conexión

Por defecto el API usa LocalDB. Si su SQL Server es otro, cambie `ConnectionStrings:GestionNotas` en `backend/src/GestionNotas.Api/appsettings.json`:

Escenario	Cadena de conexión
LocalDB (por defecto)	Server=(localdb)\MSSQLLocalDB;Database=GestionNotas;Trusted_Connection=True;TrustServerCertificate=True
SQL Server Express	Server=.\SQLEXPRESS;Database=GestionNotas;Trusted_Connection=True;TrustServerCertificate=True
Usuario y contraseña SQL	Server=MI_SERVIDOR;Database=GestionNotas;User Id=MI_USUARIO;Password=MI_CLAVE;TrustServerCertificate=True

4. API REST

En una terminal, desde la carpeta backend:

```
dotnet run --project src/GestionNotas.Api --launch-profile http
```

```
PS D:\Apps\Prueba Técnica Audisoft\backend> dotnet run --project src/GestionNotas.Api --launch-profile http
Usando la configuración de inicio de src\GestionNotas.Api\Properties\launchSettings.json...
Compilando...
```

Figura 5. Arranque del API con dotnet run.

El API está listo cuando la consola muestra `Now listening on: http://localhost:5080`.

dotnet run no abre el navegador automáticamente. Abra manualmente `http://localhost:5080`, que redirige a la documentación interactiva. Desde Visual Studio, en cambio, basta con elegir el perfil **http** y pulsar Iniciar (F5).

Dirección	Descripción
<code>http://localhost:5080/swagger</code>	Documentación interactiva (Swagger UI): permite probar cada endpoint desde el navegador.
<code>http://localhost:5080/api</code>	Base del API REST.
<code>http://localhost:5080/openapi/v1.json</code>	Especificación OpenAPI.
<code>http://localhost:5080/health</code>	Estado del API y de la conexión a la base de datos.

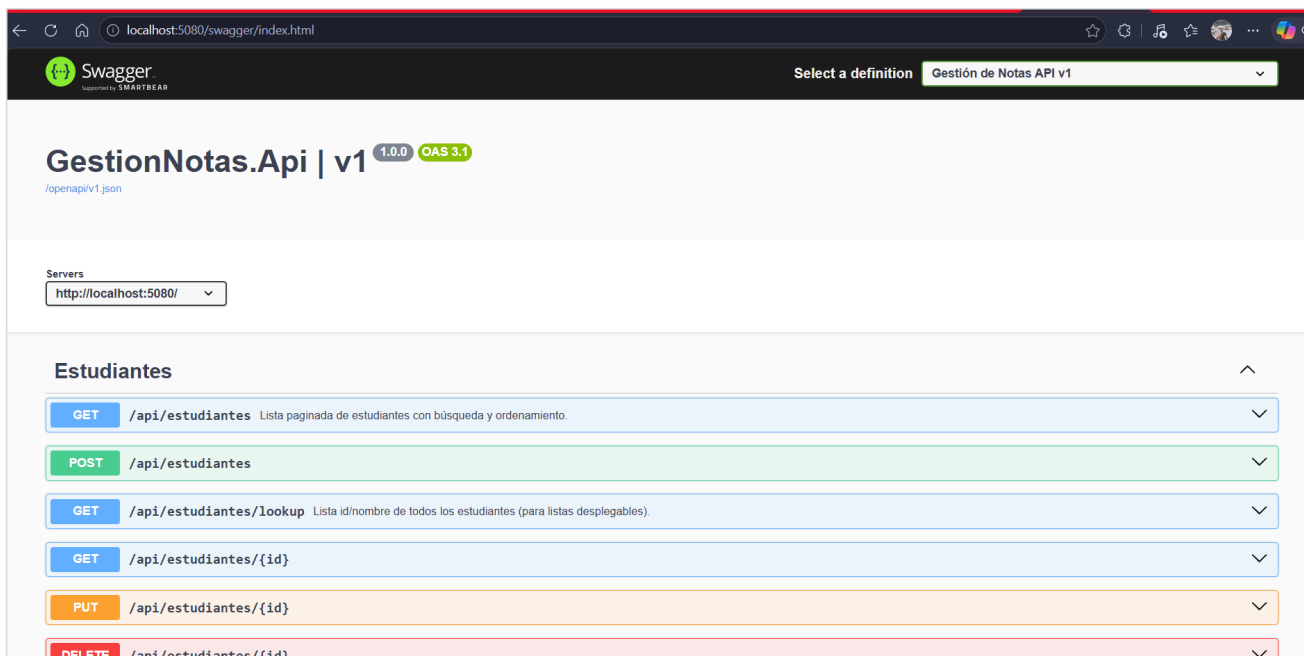


Figura 6. Swagger UI con los endpoints del API.

Endpoints

Recurso	Endpoints
Estudiantes	GET /api/estudiantes · GET /api/estudiantes/{id} · GET /api/estudiantes/lookup · POST · PUT /{id} · DELETE /{id}
Profesores	GET /api/profesores · GET /api/profesores/{id} · GET /api/profesores/lookup · POST · PUT /{id} · DELETE /{id}
Notas	GET /api/notas (filtros idEstudiante, idProfesor) · GET /{id} · POST · PUT /{id} · DELETE /{id}
Reportes	GET /api/reportes/resumen (indicadores del panel de inicio)

Los listados aceptan page, pageSize (máximo 100), search, sortBy y sortDirection (asc/desc). Los errores responden con ProblemDetails y mensajes en español: 400 (validación), 404 (no encontrado) y 409 (operación no permitida).

5. Pruebas unitarias

Desde la carpeta backend:

```
dotnet test
```

```
PS D:\Apps\Prueba Técnica Audisoft\backend> dotnet test
Restauración completada (1.1s)
  GestionNotas.Domain net10.0 realizado correctamente (0.2s) → src\GestionNotas.Domain\bin\Debug\net10.0\Ge
stionNotas.Domain.dll
  GestionNotas.Application net10.0 realizado correctamente (0.3s) → src\GestionNotas.Application\bin\Debug\
net10.0\GestionNotas.Application.dll
  GestionNotas.Infrastructure net10.0 realizado correctamente (0.4s) → src\GestionNotas.Infrastructure\bin\
Debug\net10.0\GestionNotas.Infrastructure.dll
  GestionNotas.UnitTests net10.0 realizado correctamente (1.7s) → tests\GestionNotas.UnitTests\bin\Debug\ne
t10.0\GestionNotas.UnitTests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.4+50e68bbb8b (64-bit .NET 10.0.12)
[xUnit.net 00:00:00.14]   Discovering: GestionNotas.UnitTests
[xUnit.net 00:00:00.20]   Discovered:   GestionNotas.UnitTests
[xUnit.net 00:00:00.23]   Starting:     GestionNotas.UnitTests
[xUnit.net 00:00:01.30]   Finished:     GestionNotas.UnitTests
  GestionNotas.UnitTests net10.0 realizado correctamente prueba (2.6s)

Resumen de pruebas: total: 33, con errores: 0, correcto: 33, omitido: 0, duración: 2.6 s
Compilación realizado correctamente en 6.4s
```

Figura 7. Resultado de las pruebas unitarias: 33 correctas, 0 con errores.

Las pruebas cubren las reglas de negocio de los servicios: validaciones de nombre y valor, nombres duplicados, evaluaciones repetidas, existencia de estudiante y profesor, protección al eliminar con notas asociadas, filtros y normalización de textos.

6. Frontend

Con el API en ejecución, abra **otra** terminal en la carpeta frontend:

```
cd frontend
npm install
npm start
```

1. `npm install` descarga las dependencias (solo la primera vez; tarda unos minutos).
2. `npm start` compila la aplicación, la sirve en **`http://localhost:4300`** y abre el navegador automáticamente.

```
PS D:\Apps\Prueba Técnica AudiSoft\frontend> npm start

> gestion-notas-web@1.0.0 start
> ng serve --open

Initial chunk files | Names | Raw size
main.js | main | 18.16 kB
styles.css | styles | 10.65 kB
chunk-6S7ZRSZB.js | - | 2.50 kB
chunk-TSHS5Y3X.js | - | 1.12 kB
| Initial total | 32.43 kB

Lazy chunk files | Names | Raw size
chunk-2H7OYQKG.js | notas-list-component | 45.01 kB
chunk-HQ5SRUM4.js | dashboard-component | 25.58 kB
chunk-NVMWOFCP.js | estudiantes-list-component | 23.82 kB
chunk-TKTZ6ESG.js | profesores-list-component | 21.11 kB
chunk-LAQLRWKM.js | - | 15.32 kB
chunk-L34FXXSA.js | - | 4.08 kB
chunk-QHWFN2AX.js | - | 201 bytes

Application bundle generation complete. [9.417 seconds] - 2026-09-29T20:15:07.285Z

Watch mode enabled. Watching for file changes...
NOTE: Raw file sizes do not reflect development server per-request transformations.
  → Local: http://localhost:4300/
  → press h + enter to show help
```

Figura 8. Compilación del frontend con `npm start`: aplicación disponible en `http://localhost:4300`.



Figura 9. Panel de inicio: totales, promedio general, porcentaje de aprobación, mejores estudiantes y promedio por profesor.

El frontend usa el puerto fijo **4300** para no chocar con otros proyectos Angular en el 4200. La URL del API se configura en `frontend/src/environments/environment.development.ts`.

7. Uso de la aplicación

La aplicación tiene un menú lateral con **Inicio**, **Estudiantes**, **Profesores** y **Notas**. Cada página muestra una tabla paginada en el servidor, con búsqueda y orden por columnas, y permite crear, editar y eliminar registros. Cada acción muestra una alerta con el resultado.

Estudiantes y profesores

Estudiantes
Consulte, registre y actualice la información de los estudiantes.

Buscar por nombre

ID	Nombre	Notas	Promedio	Acciones
1	Sofía Ramírez	3	3,40	
2	Santiago Gómez	3	2,97	
3	Valentina Muñoz	3	3,37	
4	Mateo Rojas	3	3,50	
5	Isabella Díaz	3	2,40	

Registros por página: 5 1 - 5 de 12

Figura 10. Listado de estudiantes: paginación en el servidor (5 registros por página, 1-5 de 12), búsqueda, cantidad de notas y promedio.

Nuevo estudiante

Nombre completo* test 4/100

Cancelar Crear

Figura 11. Formulario de creación de un estudiante.

Estudiante creado correctamente.

Nuevo estudiante

Figura 12. Alerta de confirmación: «Estudiante creado correctamente».

La página de profesores funciona igual. El nombre es obligatorio (2 a 100 caracteres) y no puede repetirse.

Notas

Notas

Calificaciones asignadas por los profesores a los estudiantes (escala de 0,0 a 5,0).

[+ Nueva nota](#)

ID	Evaluación	Estudiante	Profesor	Valor	Acciones
1	Parcial 2	Sofía Ramírez	María Fernanda López	3,5	
15	Parcial 1	Isabella Díaz	Jorge Iván Martínez	2,4	
17	Parcial 1	Samuel Torres	Jorge Iván Martínez	3,8	
18	Parcial 2	Samuel Torres	Ana Lucía Herrera	2,4	
19	Parcial 1	Mariana Vargas	Jorge Iván Martínez	3,0	
20	Parcial 2	Mariana Vargas	Ana Lucía Herrera	2,4	
21	Parcial 2	Sebastián Peña	Ana Lucía Herrera	4,2	

Registros por página: 10
1 - 7 de 7

Figura 13. Listado de notas con búsqueda por texto y filtros por estudiante y profesor. El color indica si la nota aprueba ($\geq 3,0$).

Nueva nota

Nombre de la evaluación*

Estudiante*

Profesor*

Valor (0,0 a 5,0)*

Figura 14. Formulario de nueva nota: evaluación, estudiante, profesor y valor de 0,0 a 5,0.



Figura 15. Validación del backend: el estudiante ya tiene esa evaluación con ese profesor.

Eliminación

Toda eliminación pide confirmación. Un estudiante o profesor con notas no se puede eliminar: primero deben eliminarse sus notas.



Figura 16. Confirmación antes de eliminar una nota.

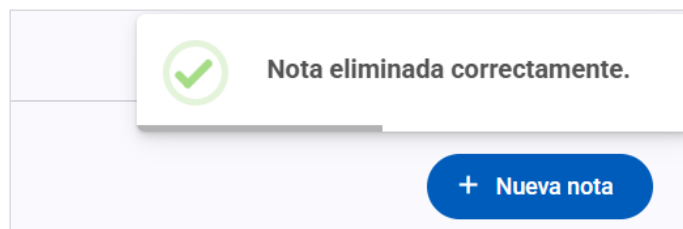


Figura 17. Alerta de confirmación: «Nota eliminada correctamente».

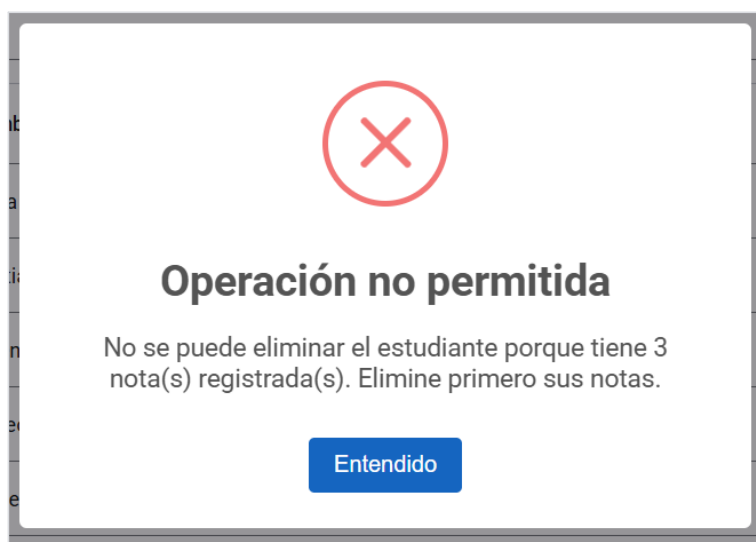


Figura 18. Protección de integridad: no se puede eliminar un estudiante con notas registradas.

8. Pruebas del API con Postman

La carpeta docs/postman/ contiene la colección y dos entornos:

Archivo	Contenido
GestionNotas.postman_collection.json	45 peticiones con pruebas automáticas.
Local.postman_environment.json	baseUrl = http://localhost:5080
Azure.postman_environment.json	baseUrl = API desplegado en Azure

1. En Postman, pulse **Import** y arrastre los tres archivos.
2. Arriba a la derecha, seleccione el entorno **Gestión de Notas — Local** (con el API en ejecución) o **— Azure**.
3. Clic derecho sobre la colección, **Run collection** y luego **Run**.

La colección crea un estudiante, un profesor y una nota; prueba el listado paginado, la búsqueda, el orden, los filtros, la edición, las validaciones (400), los recursos inexistentes (404) y la protección de integridad (409). Al final elimina lo que creó, así que puede ejecutarse las veces que se quiera sin alterar los datos.

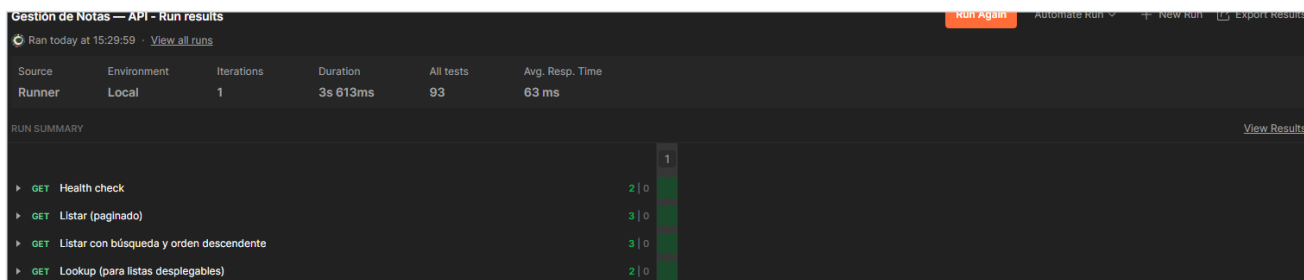


Figura 19. Ejecución de la colección en el entorno Local: 93 pruebas sin fallos.

9. Solución de problemas

Síntoma	Causa y solución
El API no arranca: «Falta la cadena de conexión» o no conecta a SQL Server.	Revise ConnectionStrings:GestionNotas en appsettings.json (sección 3). Para LocalDB, compruebe que esté instalado con <code>sqllocaldb info</code> .
Error al ejecutar el script: la base está en uso o «Database already exists».	Ejecute el script con la base master seleccionada y cierre otras ventanas conectadas a GestionNotas.
El frontend muestra «No se pudo conectar con el servidor».	El API no está en ejecución o no está en el puerto 5080. Arranque el API (sección 4) y recargue la página.
«Port 4300 is already in use» al ejecutar <code>npm start</code> .	Otro proceso usa el puerto. Ciérrelo o ejecute <code>npx ng serve --port 4400</code> .
<code>http://localhost:5080</code> muestra una página de error.	Use el perfil http : <code>dotnet run --project src/GestionNotas.Api --launch-profile http</code> .
La demostración en línea tarda en cargar.	Es normal la primera vez: el API y la base de datos se encienden bajo demanda.